AI Token Observatory

Guia do projeto · extensão VS Code open source (MIT) · v0.1.0 · outubro/2026

1. O que é

Extensão local-first que mostra **tokens, custos e requests** dos agentes de IA usados no VS Code — **Claude Code, Cline, Roo Code e Kilo Code** — lendo os arquivos que essas ferramentas já gravam no disco. Sem conta, sem telemetria, sem rede.

- **Para quem:** devs que querem saber quanto e onde estão gastando com agentes de IA.
- **Como:** descobre as pastas → lê incrementalmente → indexa num SQLite local → exibe num painel React.
- **Entrega:** arquivo `ai-token-observatory.vsix` (~818 KB).

2. Instalação e uso rápido

```
code --install-extension ai-token-observatory.vsix
```

1. Recarregue a janela e clique no ícone **AI Token Observatory** na Activity Bar.
2. A primeira sincronização roda sozinha; depois, watchers + polling mantêm o índice atualizado.
3. Use os filtros globais (período, ferramenta, provedor, modelo, projeto, sessão). Clicar num gráfico também filtra.

Comando (Ctrl+Shift+P)	Função
AI Token Observatory: Open Dashboard	Abre o painel principal
AI Token Observatory: Rescan Sources	Redescobre as pastas das ferramentas
AI Token Observatory: Refresh Data	Força sincronização incremental
AI Token Observatory: Export Report	Exporta as requests em CSV ou JSON
AI Token Observatory: Show Diagnostics	Mostra a saúde das fontes e os erros de leitura

3. Fontes suportadas

Ferramenta	Onde lê	Unidade
Claude Code	`~/.claude/projects/**/*.jsonl` ou `CLAUDE_CONFIG_DIR`	1 mensagem do assistente (dedup `message.id:requestId`)
Cline (legado)	`globalStorage/saoudrizwan.claude-dev/tasks/*/ui_messages.json`	1 `api_req_started` concluído
Cline v2	`~/.cline/data/sessions/*/*.messages.json`	métricas por mensagem
Roo Code	`globalStorage/rooveterinaryinc.roo-cline/tasks`	igual ao Cline legado
Kilo Code	`globalStorage/kilocode.kilo-code/tasks`	igual ao Cline legado

O `globalStorage` é procurado em VS Code, Insiders, VSCodium, Cursor e Windsurf. Arquivos de segredo (ex.: `secrets.json`) **nunca** são lidos.

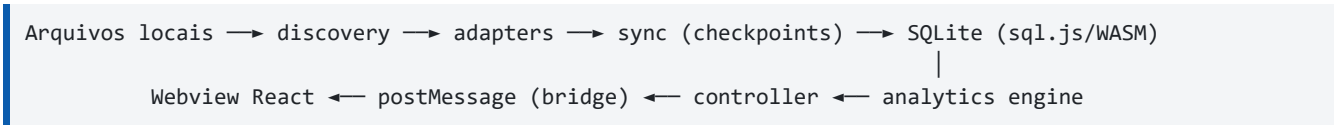
4. Telas do painel

Tela	Conteúdo
Overview	KPIs com comparação ao período anterior, séries temporais, mix de tokens, top modelos
Requests	Tabela ordenável/pesquisável, colunas configuráveis, 50/página; drawer de detalhe
Sessions	Lista de sessões e linha do tempo de cada uma

Projects / Models / Costs	Agregações por dimensão; custo por base (reportado/estimado/desconhecido)
Insights	Anomalias, picos, outliers, eficiência de cache, horário de pico, previsão do mês
Integrations	Saúde de cada fonte: arquivos, requests, erros, limitações
Settings	Pastas manuais, preços customizados, preview, intervalo de polling

Também há **tree view** lateral e **Status Bar** com o custo da sessão ativa.

5. Arquitetura



Pasta	Responsabilidade
`src/core`	Tipos (`RawRequest` , `AIRequest` , `Adapter`), utilitários de FS, provedores
`src/discovery`	Detecta pastas de cada ferramenta + pastas manuais
`src/adapters`	`claudeCode` , `taskFolder` (Cline legado/Roo/Kilo), `clineV2`
`src/sync`	Leitura incremental, checkpoint por arquivo, status de saúde
`src/storage`	Índice SQLite no `globalStorage` , upsert, recuperação de corrupção
`src/pricing`	Tabela de preços versionada, estimativa, total de tokens
`src/analytics`	Filtros, agregações, estatística, insights, forecast, payload do painel
`src/export` , `preview`	CSV/JSON seguros; preview sob demanda do texto original
`src/panel` , `controller` , `extension`	Webview com CSP+nonce, mensagens, tree view, status bar
`webview/`	React + Zustand + Tailwind + ECharts, build Vite (IIFE)

6. Regras de dados

Identidade e deduplicação

- ID: `${sourceTool}:${sourceRecordId}` , gravado por upsert — ressincronizar nunca duplica.
- Claude Code: várias linhas de streaming da mesma mensagem → vale a **última**. `<synthetic>` é ignorado.
- Totais de tarefa/sessão **nunca** são somados aos registros por request.

Leitura incremental

- Checkpoint guarda tamanho, mtime e offset. JSONL continua do offset; linha final incompleta fica para a próxima rodada.
- Arquivo menor que o offset (truncado/rotacionado) → relê do início e substitui os registros.
- Fontes JSON inteiras (Cline/Roo/Kilo) são relidas e substituídas por arquivo.

Custo — 3 bases, nesta ordem

Base	Quando
Reportado	A ferramenta gravou o custo (Cline/Roo/Kilo). Zero reportado continua 0.
Estimado	Modelo na tabela `v1-2026-01` ou em preços customizados. Ollama/LM Studio = 0.
Desconhecido	Sem dado confiável → `null` . Nunca é inventado.

> Uso do Claude Code por assinatura aparece como **equivalente em API**, não como cobrança real.

Total de tokens = input + output + cache read + cache write + reasoning. Campo ausente = desconhecido (≠ 0).

Insights e previsão

- Anomalias por mediana/MAD com z robusto > 3,5.
- Amostra mínima: 10 requests, 5 sessões, 7 dias.
- Previsão do mês: regressão linear diária com faixa de ±1,96·SE.

7. Privacidade e segurança

- Só campos de uso são indexados; **prompts e respostas não são armazenados**.
- Preview do texto original é opcional (desligado por padrão), lido direto da fonte e só dentro das pastas conhecidas.
- Webview com Content Security Policy restrita e nonce; sem scripts externos.
- Export CSV protege contra injeção de fórmula; caminhos de workspace só saem se solicitado.
- Nenhuma chamada de rede, nenhuma telemetria.

8. Configurações

Chave (`aiTokenObservatory.*`)	Padrão	Uso
`refreshIntervalSeconds`	30	Intervalo do polling (complementa os file watchers)
`manualSourceFolders`	[]	Pastas extras para ler
`showStatusBar`	true	Mostra o custo da sessão ativa na Status Bar
`allowContentPreview`	false	Permite preview do conteúdo original (nunca persistido)
`brlRate` / `brlRateDate`	0 / ""	Cotação USD→BRL opcional (0 desativa)
`customPricing`	{}	USD por milhão de tokens por modelo

```
"aiTokenObservatory.customPricing": {
  "cline-pass/deepseek-v4-pro": { "input": 0.5, "output": 2, "cacheRead": 0.05, "cacheWrite": 0.5 }
}
```

9. Desenvolvimento

```
npm install
npm run typecheck        # tsc --noEmit
npm test                 # vitest: 23 testes
npm run build            # dist/extension.js, dist/sql-wasm.wasm, dist/webview/*
npx @vscode/vsce package --no-dependencies --out ai-token-observatory.vsix
```

- F5 abre o Extension Development Host.
- `scripts/smoke-real.ts` : valida contra os dados reais da máquina (somente leitura, índice em memória).
- Testes usam fixtures sintéticas (`test/fixtures.ts`) — nunca commitar prompts reais.

Adicionar uma nova fonte

1. Implementar `Adapter` (`listFiles` , `parseFile` , `incremental`) retornando `RawRequest` com `null` no que a fonte não registra.
2. Registrar a detecção em `src/discovery/discovery.ts` e o mapeamento em `src/sync/sync.ts` .
3. Criar testes com fixtures cobrindo dedup, arquivo parcial e truncamento.

10. Limitações conhecidas

- Modelos fora da tabela (ex.: `copilot/*` , `cline-pass/*`) ficam com custo desconhecido até receberem preço customizado.
- Tabela de preços estática; precisa de atualização manual.
- Claude Code não registra duração; em Cline/Roo ela é aproximada pela mensagem seguinte.
- Kilo Code validado apenas com fixtures.
- Sem testes de integração dentro do VS Code (`@vscode/test-electron`).
- Bundle do painel ~1,2 MB (ECharts completo).
- O `vsce` local está quebrado; empacotar via `npx @vscode/vsce` .